

CS7015 (Deep Learning) : Lecture 13

Visualizing Convolutional Neural Networks, Guided Backpropagation, Deep Dream, Deep Art, Fooling Convolutional Neural Networks

CS7015 (Deep Learning) : Lecture 13

Visualizing Convolutional Neural Networks, Guided Backpropagation, Deep Dream, Deep Art, Fooling Convolutional Neural Networks

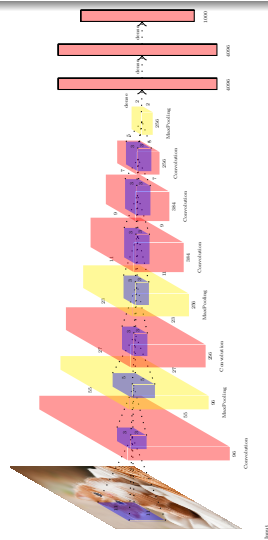
Mitesh M. Khapra

Department of Computer Science and Engineering
Indian Institute of Technology Madras

Acknowledgements

- Andrej Karpathy Video Lecture on Visualization and Deep Dream*

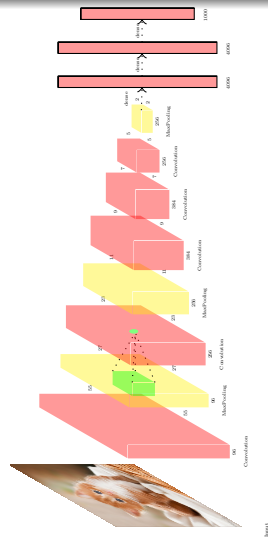
*Visualization, Deep Dream, Neural Style, Adversarial Examples



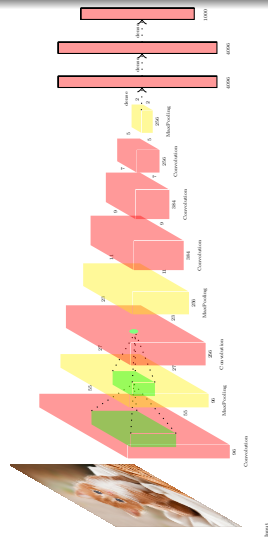
- Consider some neurons in a given layer of a CNN



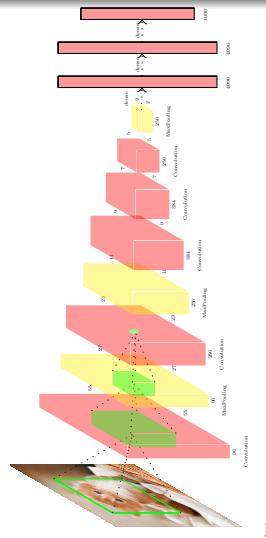
- Consider some neurons in a given layer of a CNN
- We can feed in images to this CNN and identify the images which cause these neurons to fire



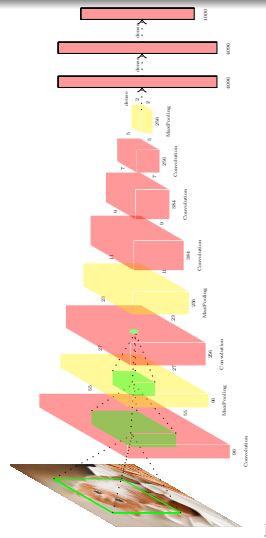
- Consider some neurons in a given layer of a CNN
- We can feed in images to this CNN and identify the images which cause these neurons to fire
- We can then trace back to the patch in the image which causes these neurons to fire



- Consider some neurons in a given layer of a CNN
- We can feed in images to this CNN and identify the images which cause these neurons to fire
- We can then trace back to the patch in the image which causes these neurons to fire



- Consider some neurons in a given layer of a CNN
- We can feed in images to this CNN and identify the images which cause these neurons to fire
- We can then trace back to the patch in the image which causes these neurons to fire



- Consider some neurons in a given layer of a CNN
- We can feed in images to this CNN and identify the images which cause these neurons to fire
- We can then trace back to the patch in the image which causes these neurons to fire
- Let us look at the result of one of such experiment conducted by Grishick et al., 2014

- They consider 6 neurons in the pool5 layer and find the image patches which cause these neurons to fire
- One neuron fires for people faces



- They consider 6 neurons in the pool5 layer and find the image patches which cause these neurons to fire
- Another neuron fires for dog faces



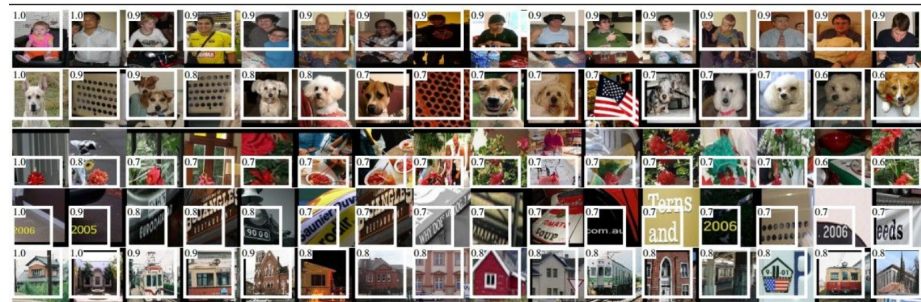
- They consider 6 neurons in the pool5 layer and find the image patches which cause these neurons to fire
- Another neuron fires for flowers



- They consider 6 neurons in the pool5 layer and find the image patches which cause these neurons to fire
- Another neuron fires for numbers

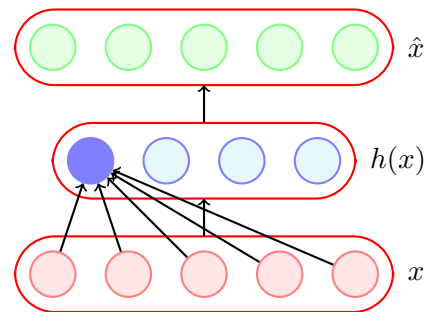


- They consider 6 neurons in the pool5 layer and find the image patches which cause these neurons to fire
- Another neuron fires for houses



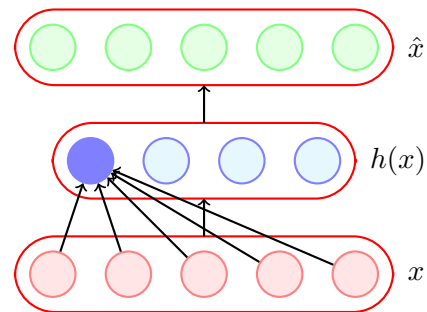
- They consider 6 neurons in the pool5 layer and find the image patches which cause these neurons to fire
- Another neuron fires for shiny surfaces





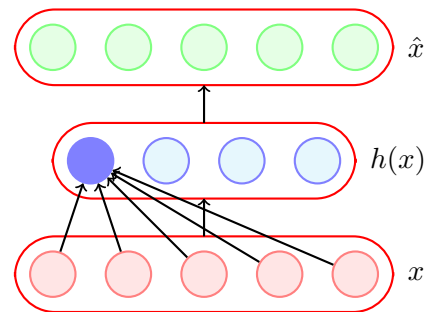
- Recall that we had done something similar while discussing autoencoders

$$\begin{aligned} & \max_x \{w^T x\} \\ & s.t. \quad ||x||^2 = x^T x = 1 \\ & \text{Solution: } x = \frac{w_1}{\sqrt{w_1^T w_1}} \end{aligned}$$



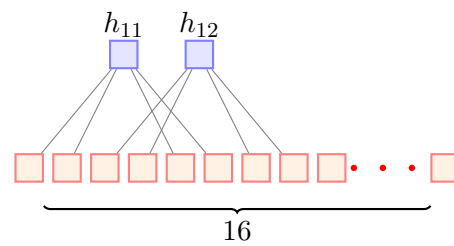
- Recall that we had done something similar while discussing autoencoders
- We are interested in finding an input which maximally excites a neuron

$$\begin{aligned} & \max_x \{w^T x\} \\ & s.t. \quad ||x||^2 = x^T x = 1 \\ & \text{Solution: } x = \frac{w_1}{\sqrt{w_1^T w_1}} \end{aligned}$$

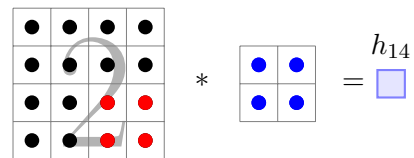


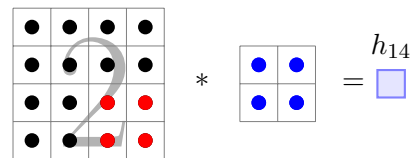
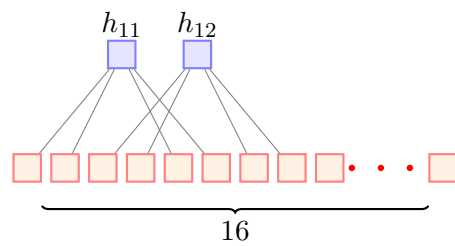
- Recall that we had done something similar while discussing autoencoders
- We are interested in finding an input which maximally excites a neuron
- Turns out that the input which will maximally activate a neuron is $\frac{W}{\|W\|}$

$$\begin{aligned} & \max_x \{w^T x\} \\ & s.t. \quad \|x\|^2 = x^T x = 1 \\ & \text{Solution: } x = \frac{w_1}{\sqrt{w_1^T w_1}} \end{aligned}$$

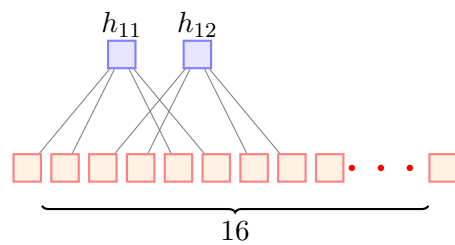


- Now recall that we can think of a CNN also as a feed-forward network with sparse connections and weight sharing



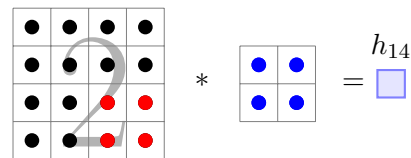
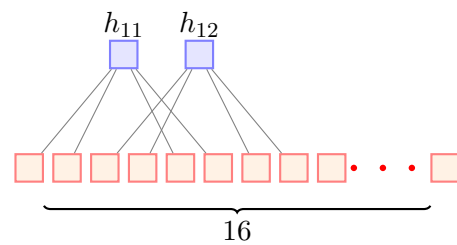


- Now recall that we can think of a CNN also as a feed-forward network with sparse connections and weight sharing
- Once again, we are interested in knowing what kind of inputs will cause a given neuron to fire

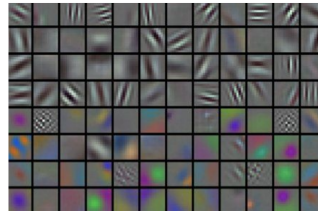


$$\begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} * \begin{array}{|c|c|} \hline \bullet & \bullet \\ \hline \bullet & \bullet \\ \hline \end{array} = h_{14}$$

- Now recall that we can think of a CNN also as a feed-forward network with sparse connections and weight sharing
- Once again, we are interested in knowing what kind of inputs will cause a given neuron to fire
- The solution would be the same ($\frac{W}{\|W\|}$) where W is the filter (2×2 , in this case)

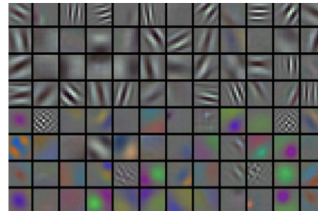


- Now recall that we can think of a CNN also as a feed-forward network with sparse connections and weight sharing
- Once again, we are interested in knowing what kind of inputs will cause a given neuron to fire
- The solution would be the same ($\frac{W}{\|W\|}$) where W is the filter (2×2 , in this case)
- We can thus think of these filters as pattern detectors



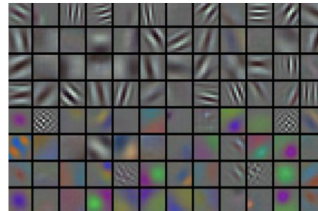
- We can simply plot the $K \times K$ weights (filters) as images & visualize them as patterns

$$\begin{aligned} & \max_x \{w^T x\} \\ & s.t. \quad ||x||^2 = x^T x = 1 \\ \text{Solution: } & x = \frac{w_1}{\sqrt{w_1^T w_1}} \end{aligned}$$



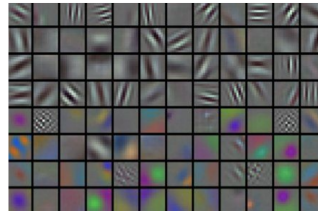
- We can simply plot the $K \times K$ weights (filters) as images & visualize them as patterns
- The filters essentially detect these patterns (by causing the neurons to maximally fire)

$$\begin{aligned} & \max_x \{w^T x\} \\ & s.t. \quad ||x||^2 = x^T x = 1 \\ \text{Solution: } & x = \frac{w_1}{\sqrt{w_1^T w_1}} \end{aligned}$$



$$\begin{aligned} & \max_x \{w^T x\} \\ & s.t. \quad ||x||^2 = x^T x = 1 \\ \text{Solution: } & x = \frac{w_1}{\sqrt{w_1^T w_1}} \end{aligned}$$

- We can simply plot the $K \times K$ weights (filters) as images & visualize them as patterns
- The filters essentially detect these patterns (by causing the neurons to maximally fire)
- This is only interpretable for the filters in the first convolution layer



$$\begin{aligned} \max_x \quad & \{w^T x\} \\ \text{s.t.} \quad & \|x\|^2 = x^T x = 1 \end{aligned}$$

Solution: $x = \frac{w_1}{\sqrt{w_1^T w_1}}$

- We can simply plot the $K \times K$ weights (filters) as images & visualize them as patterns
- The filters essentially detect these patterns (by causing the neurons to maximally fire)
- This is only interpretable for the filters in the first convolution layer (Why?)

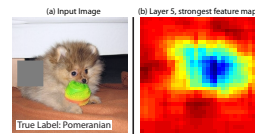
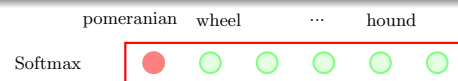
pomeranian wheel ... hound

Softmax

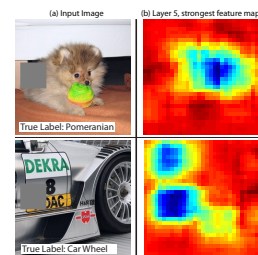
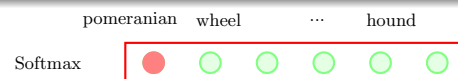
● ● ● ● ● ●



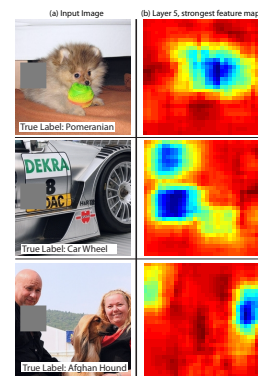
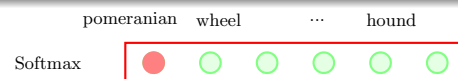
- Typically we are interested in understanding which portions of the image are responsible for maximizing the probability of a certain class



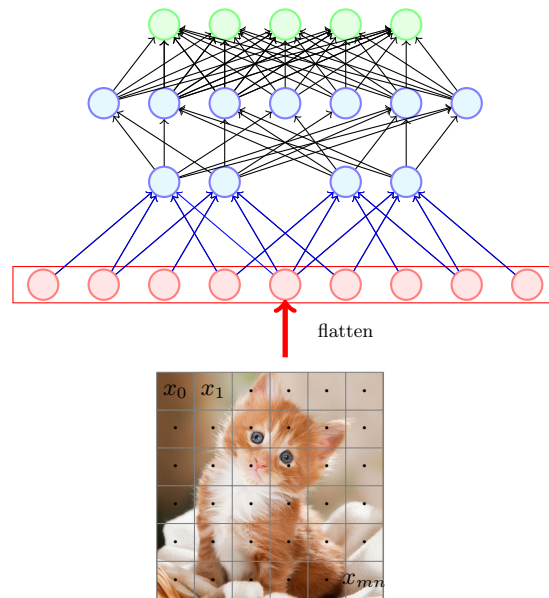
- Typically we are interested in understanding which portions of the image are responsible for maximizing the probability of a certain class
- We could occlude (gray out) different patches in the image and see the effect on the predicted probability of the correct class



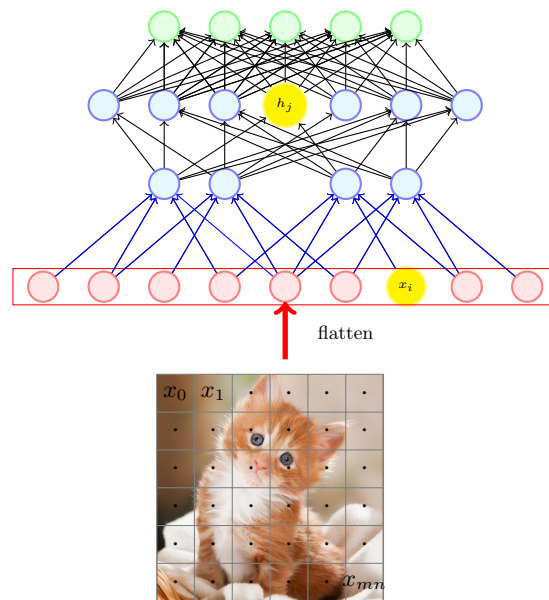
- Typically we are interested in understanding which portions of the image are responsible for maximizing the probability of a certain class
- We could occlude (gray out) different patches in the image and see the effect on the predicted probability of the correct class
- For example this heat map shows that occluding the face of the dog causes a maximum drop in the prediction probability



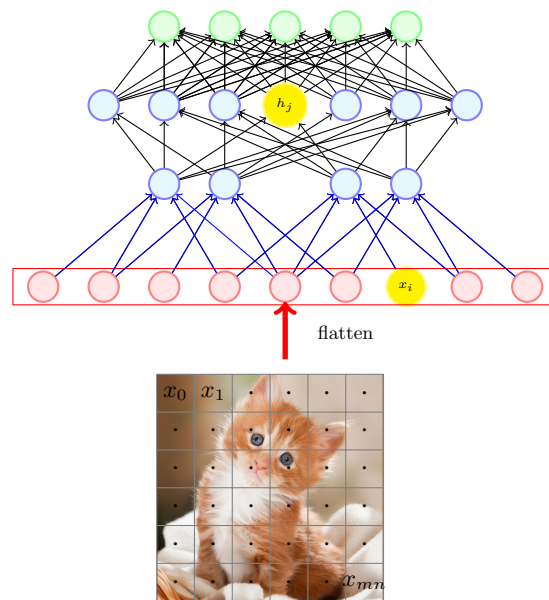
- Typically we are interested in understanding which portions of the image are responsible for maximizing the probability of a certain class
- We could occlude (gray out) different patches in the image and see the effect on the predicted probability of the correct class
- For example this heat map shows that occluding the face of the dog causes a maximum drop in the prediction probability
- Similar observations are made for other images



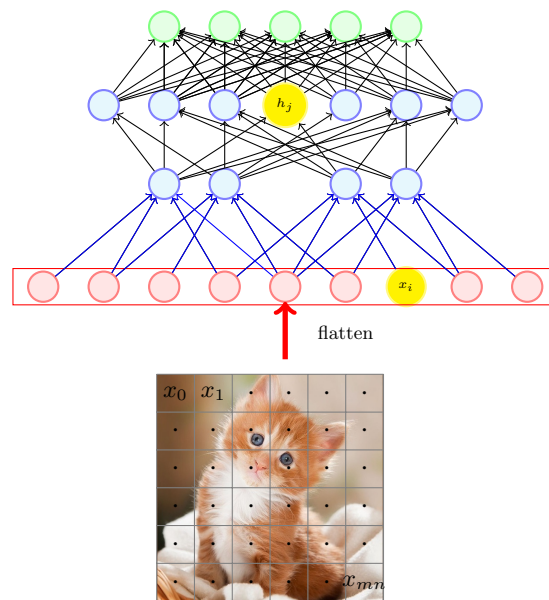
- We can think of an image as a $m \times n$ inputs $x_0, x_1, \dots, x_{m \times n}$



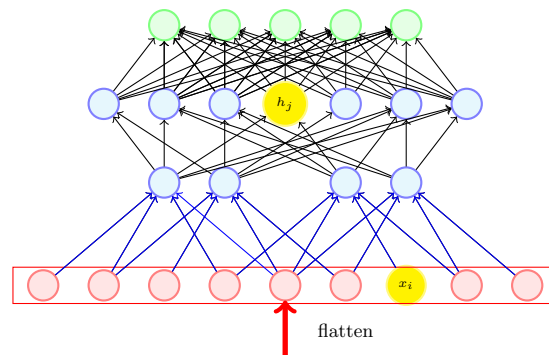
- We can think of an image as a $m \times n$ inputs $x_0, x_1, \dots, x_{m \times n}$
- We are interested in finding the influence of each of these inputs(x_i) on a given neuron(h_j)



- We can think of an image as a $m \times n$ inputs $x_0, x_1, \dots, x_{m \times n}$
- We are interested in finding the influence of each of these inputs(x_i) on a given neuron(h_j)
- If a small change in x_i causes a large change in h_j then we can say that x_i has a lot of influence of h_j

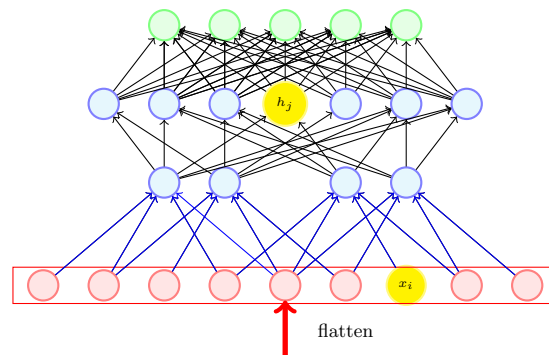


- We can think of an image as a $m \times n$ inputs $x_0, x_1, \dots, x_{m \times n}$
- We are interested in finding the influence of each of these inputs(x_i) on a given neuron(h_j)
- If a small change in x_i causes a large change in h_j then we can say that x_i has a lot of influence of h_j
- In other words the gradient $\frac{\partial h_j}{\partial x_i}$ could tell us about the influence



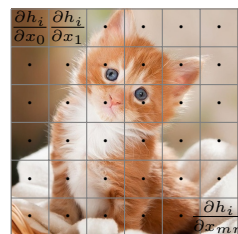
$$\frac{\partial h_j}{\partial x_i} = 0 \quad \rightarrow \text{no influence}$$

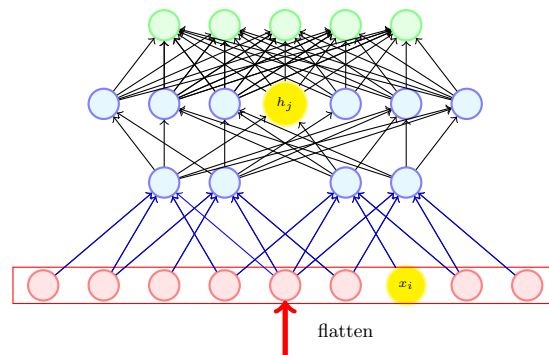




$$\frac{\partial h_j}{\partial x_i} = 0 \quad \rightarrow \text{no influence}$$

$$\frac{\partial h_j}{\partial x_i} = \text{large} \quad \rightarrow \text{high influence}$$

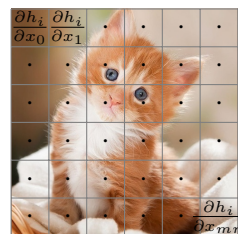


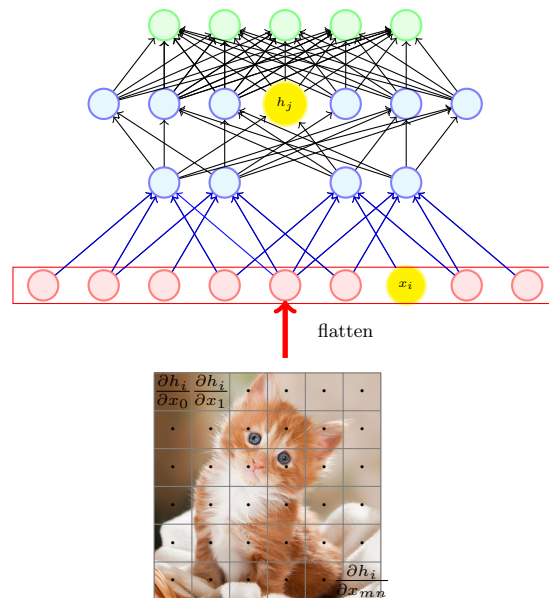


$$\frac{\partial h_j}{\partial x_i} = 0 \quad \longrightarrow \text{no influence}$$

$$\frac{\partial h_j}{\partial x_i} = \text{large} \quad \longrightarrow \text{high influence}$$

$$\frac{\partial h_j}{\partial x_i} = \text{small} \quad \longrightarrow \text{low influence}$$



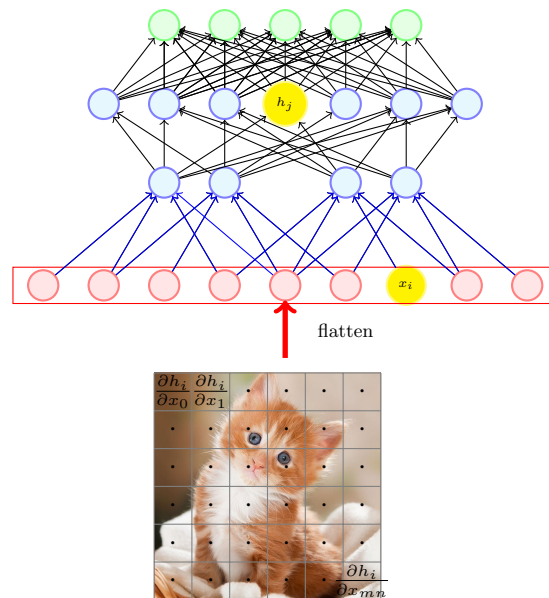


$$\frac{\partial h_j}{\partial x_i} = 0 \quad \rightarrow \text{no influence}$$

$$\frac{\partial h_j}{\partial x_i} = \text{large} \quad \rightarrow \text{high influence}$$

$$\frac{\partial h_j}{\partial x_i} = \text{small} \quad \rightarrow \text{low influence}$$

- We could just compute these partial derivatives w.r.t all the inputs

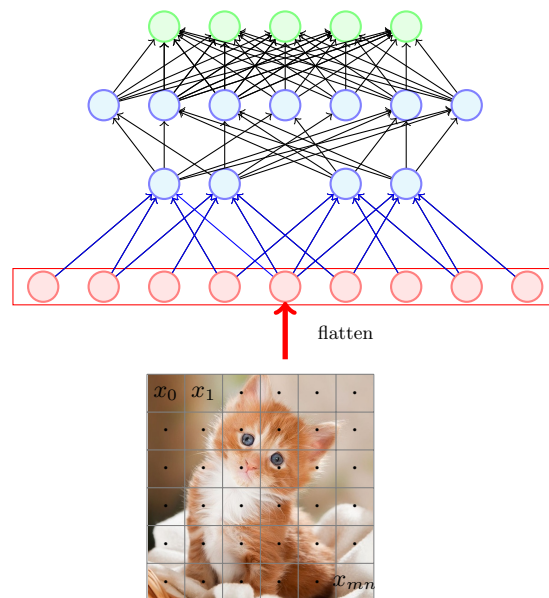


$$\frac{\partial h_j}{\partial x_i} = 0 \quad \longrightarrow \text{no influence}$$

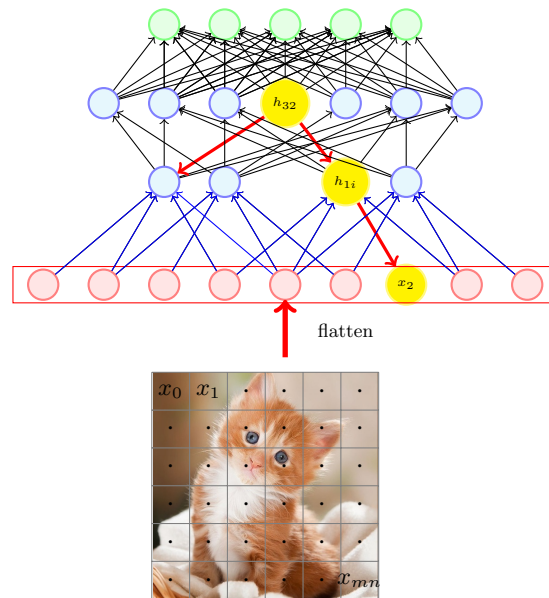
$$\frac{\partial h_j}{\partial x_i} = \text{large} \quad \longrightarrow \text{high influence}$$

$$\frac{\partial h_j}{\partial x_i} = \text{small} \quad \longrightarrow \text{low influence}$$

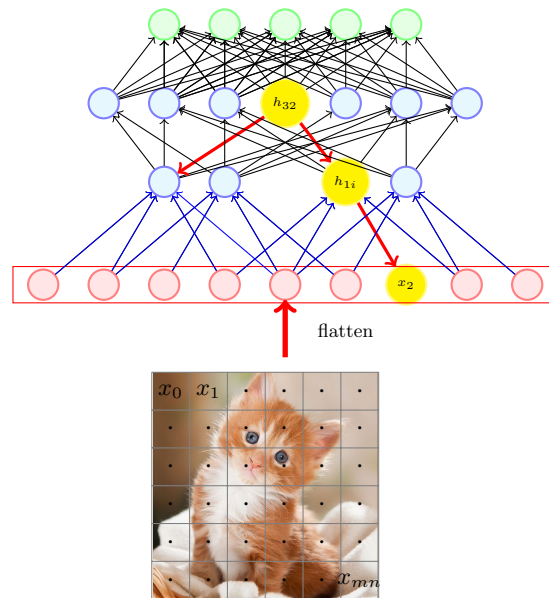
- We could just compute these partial derivatives w.r.t all the inputs
- And then visualize this gradient matrix as an image itself



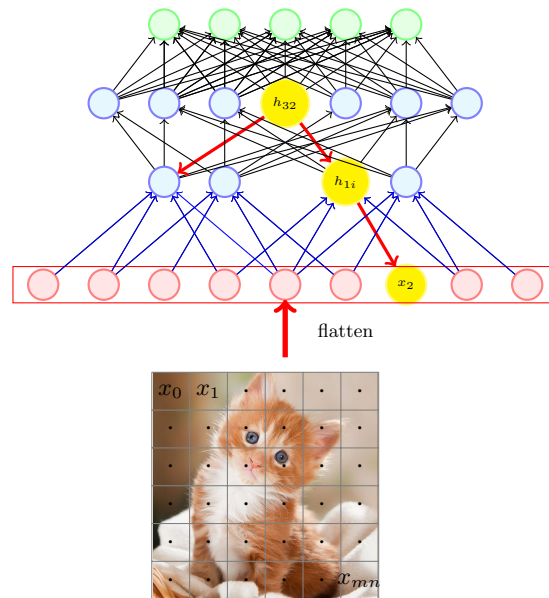
- But how do we compute these gradients?



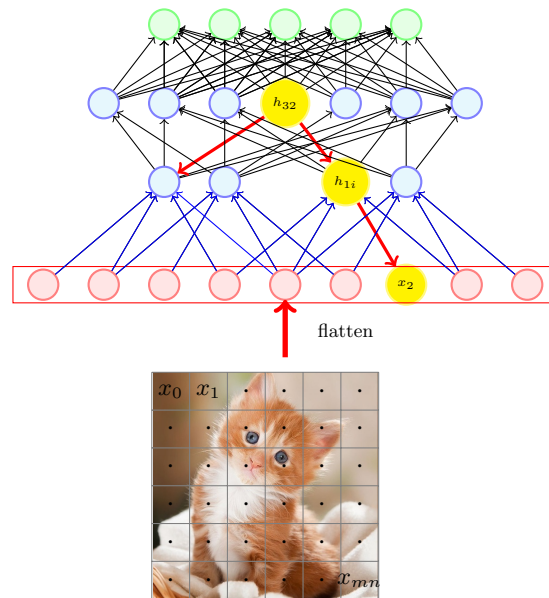
- But how do we compute these gradients?
- Recall that we can represent CNNs by feedforward neural network



- But how do we compute these gradients?
- Recall that we can represent CNNs by feedforward neural network
- Then we already know how to compute influences (gradient) using back-propagation



- But how do we compute these gradients?
- Recall that we can represent CNNs by feedforward neural network
- Then we already know how to compute influences (gradient) using back-propagation
- For example, we know how to back-prop the gradients till the first hidden layer



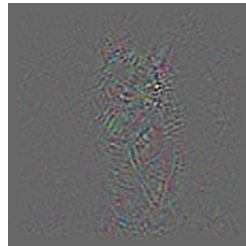
- But how do we compute these gradients?
- Recall that we can represent CNNs by feedforward neural network
- Then we already know how to compute influences (gradient) using backpropagation
- For example, we know how to backprop the gradients till the first hidden layer

$$\frac{\partial h_{32}}{\partial x_2} = \sum_{i=1}^3 \frac{\partial h_{32}}{\partial h_{1i}} \frac{\partial h_{1i}}{\partial x_2}$$

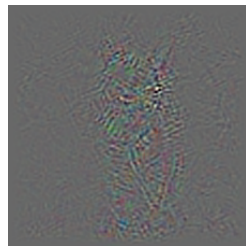
$$h_{1i} = \sum_{j=1}^4 w_{ji} x_j$$

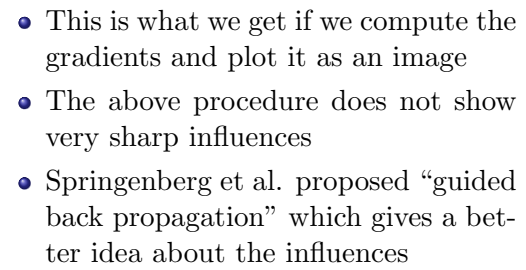
$$\frac{\partial h_{1i}}{\partial x_2} = w_{12}$$

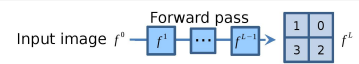
- This is what we get if we compute the gradients and plot it as an image



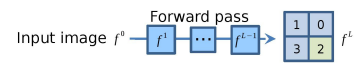
-
- This is what we get if we compute the gradients and plot it as an image
 - The above procedure does not show very sharp influences



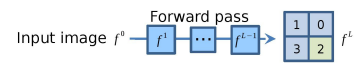




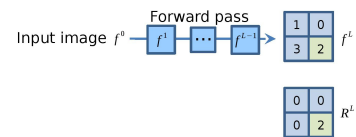
- We feed an input to the CNN and do a forward pass



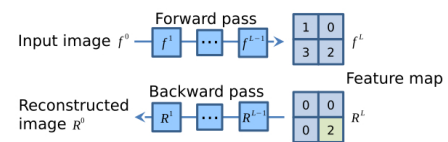
- We feed an input to the CNN and do a forward pass
- We consider one neuron in some feature map at some layer



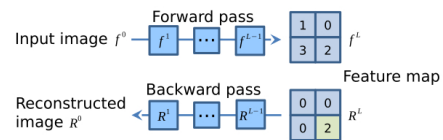
- We feed an input to the CNN and do a forward pass
- We consider one neuron in some feature map at some layer
- We are interested in finding the influence of the input on this neuron



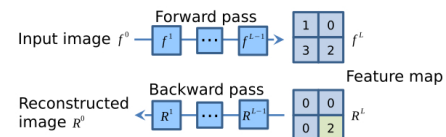
- We feed an input to the CNN and do a forward pass
- We consider one neuron in some feature map at some layer
- We are interested in finding the influence of the input on this neuron
- We retain this neuron and set all other neurons in the layer to zero



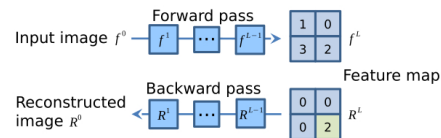
- We now backpropagate all the way to the inputs



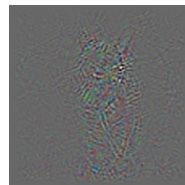
- We now backpropagate all the way to the inputs
- Recall that during forward pass relu activation allows only positive values to pass & clamps $-ve$ values to zero



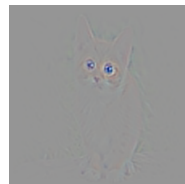
- We now backpropagate all the way to the inputs
- Recall that during forward pass relu activation allows only positive values to pass & clamps *-ve* values to zero
- Similarly during backward pass no gradient passes through the dead relu neurons



- We now backpropagate all the way to the inputs
- Recall that during forward pass relu activation allows only positive values to pass & clamps *-ve* values to zero
- Similarly during backward pass no gradient passes through the dead relu neurons
- In guided back propagation any -ve gradients flowing from the upper layer are also set to 0



Backpropagation



Guided Backpropagation

- **Intuition:** Neglect all the negative influences (gradients) and focus only on the positive influences (gradients)
- This gives a better picture of the true influence of the input